

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
Кафедра телекоммуникационных систем и вычислительных средств (ТС и ВС)

ОТЧЁТ ПО РАСЧЁТНО-ГРАФИЧЕСКОМУ ЗАДАНИЮ

по дисциплине
«Программирование»

ПРОСТЕЙШИЙ ПЕРЕВОДЧИК НА ЯЗЫКЕ С

Выполнил:
Студент группы ИКС-531
Буксанов Д. К.

Проверил:
Преподаватель
А. И. Вейлер

Новосибирск, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ПОСТАНОВКА ЗАДАЧИ	4
2 АНАЛИЗ ЗАДАЧИ	5
2.1 Методы и алгоритмы	5
2.2 Алгоритм работы программы	5
3 РЕАЛИЗАЦИЯ	7
3.1 Структура проекта	7
3.2 Ключевые структуры данных	7
3.3 Модули управления регистром	7
4 ТЕСТИРОВАНИЕ	9
4.1 Запуск программы	9
4.2 Результаты работы	9
4.3 Сборка через CMake	10
ЗАКЛЮЧЕНИЕ	11
ПРИЛОЖЕНИЕ А. ЛИСТИНГ ПРОГРАММЫ	12

ВВЕДЕНИЕ

Целью расчётно-графического задания по курсу «Программирование» является разработка консольной программы на языке C для автоматизированного контекстного перевода текстовых файлов.

Программа считывает исходный текст, выполняет его лексический анализ и переводит слова в соответствии с предоставленным внешним словарем, строго сохраняя исходное форматирование документа, включая авторскую пунктуацию, отступы и переносы строк.

Задачи, решённые в ходе работы:

- разработка парсера текстовых файлов с посимвольным анализом;
- динамическое управление оперативной памятью для загрузки словарей неограниченного размера (`malloc/realloc`);
- использование пользовательских структур данных (`struct`) для поддержки многозначных слов;
- реализация алгоритмов побайтового анализа для сохранения регистра букв (поддержка ASCII и UTF-8 кириллицы);
- сборка проекта через систему автоматизации CMake.

1 ПОСТАНОВКА ЗАДАЧИ

Разработать программу *translate*, выполняющую перевод текста с помощью словаря. Программа принимает на вход три файла:

1. Первый содержит исходный текст на русском языке.
2. Второй — файл словаря (каждому слову на исходном языке соответствует слово на целевом).
3. Третий файл создаётся, и в него записывается результат перевода.

Особые условия: Форматирование исходного текста (отступы, переносы строк, знаки препинания) должно быть сохранено. Замена происходит только по полному соответствию слова.

Критерии реализации:

- полное соответствие заданию (словарь из файла, сохранение форматирования);
- использование динамической памяти;
- использование структур;
- сохранение исходного регистра;
- поддержка многозначных слов.

2 АНАЛИЗ ЗАДАЧИ

2.1 Методы и алгоритмы

В программе используются следующие алгоритмы и подходы:

- **Динамическое выделение памяти:** Использование функций `malloc` и `realloc` для масштабирования массива структур при загрузке словаря. Механизм включает проверку успешности выделения и использует промежуточный указатель `temp` во избежание утечек памяти при переполнении.
- **Посимвольный анализ текста:** Для стопроцентного сохранения авторского форматирования применяется потоковое чтение через `fgetc`. Буквы (ASCII и UTF-8 ≥ 128) накапливаются в строковый токен, а разделители немедленно отправляются в выходной поток без обработки словарем.
- **Токенизация строк:** Применение функции `strtok` для разделения строк файла словаря. Алгоритм осуществляет поиск дубликатов: если слово уже существует в словаре, новые варианты перевода добавляются к существующей структуре.
- **Побайтовая работа с кодировкой UTF-8:** В современных системах кириллица кодируется двумя байтами. Реализованы кастомные математические сдвиги байтов (для базовых смещений `0xD0` и `0xD1`), позволяющие корректно переводить русские символы в верхний и нижний регистр.

2.2 Алгоритм работы программы

Основной алгоритм функционирования утилиты:

1. Валидация аргументов командной строки (проверка путей к 3 файлам).
2. Открытие файла словаря и динамическая загрузка пар в массив структур.
3. Открытие файла исходного текста на чтение и файла результата на запись.

4. Цикл посимвольного чтения (пока не встречен EOF):
 - Если символ буквенный — накопление в буфер.
 - Если встречен знак препинания или пробел — завершение формирования слова, отправка его на поиск в словарь.
 - Запись найденного перевода (с учетом регистра) и исходного знака препинания в итоговый файл.
5. Очистка занятой оперативной памяти (`free`) и закрытие файловых дескрипторов.

3 РЕАЛИЗАЦИЯ

3.1 Структура проекта

Проект имеет следующую структуру:

```
/
├─ CMakeLists.txt      # Файл конфигурации сборки
├─ rgr.c               # Основной исходный код программы
├─ текст.txt           # Тестовый файл с исходным текстом
├─ словарь.txt         # Файл со словарной базой
└─ результат.txt       # Файл для записи результата перевода
```

3.2 Ключевые структуры данных

Для хранения словаря была спроектирована структура `DictRec`, связывающая оригинальное слово со спектром его возможных переводов.

```
1 typedef struct {
2     char og[256];          // Оригинальное слово ключ (поиска в нижнем
                             регистре)
3     char transl[5][256];   // Двумерный массив: до 5 вариантов перевода
4     int t_count;          // Фактическое количество добавленных переводов
5 } DictRec;
```

Данный подход позволяет хранить до 5 альтернативных значений для каждого термина. При выводе перевода программа конкатенирует эти значения через разделитель `/`.

3.3 Модули управления регистром

Программа логически разделена на компоненты обработки. Для работы с регистром используются две функции:

- `make_low(const char* src, char* dst)` — функция лексического анализа. Проверяет первый байт слова. При обнаружении заглавной буквы (латиница или кириллица UTF-8) модифицирует байты для приведения к строчному виду и возвращает флаг `is_upper = 1`.
- `fprint_upper(FILE *out, const char *str)` — функция вывода. Принимает переведенное слово из словаря и искусственно под-

нимает регистр первой буквы перед записью в FILE *out, выполняя обратный математический сдвиг.

4 ТЕСТИРОВАНИЕ

4.1 Запуск программы

Запуск программы осуществляется через интерфейс командной строки с передачей трёх позиционных аргументов.

```
./rgr текст.txt словарь.txt результат.txt
```

Программа осуществляет строгую валидацию входных параметров. В случае отсутствия файлов, нехватки прав доступа или передачи неверного числа аргументов, в стандартный поток выводится соответствующее сообщение об ошибке, и выполнение корректно завершается.

4.2 Результаты работы

Для проверки работоспособности программы был сформирован тестовый словарь и входной текст с различными знаками препинания и регистрами.

Содержимое файла `словарь.txt`:

```
привет hello hi
мир world peace
яблоко apple
замок lock castle
на on
столе table
старый old
это this
лежит lies
```

Содержимое файла `текст.txt`:

```
Привет, мир! Яблоко лежит на столе.
Это старый замок.
```

Содержимое итогового файла `результат.txt`:

```
Hello/Hi, world/peace! Apple lies on table.
This old lock/castle.
```

Анализ логов показывает, что многозначные слова успешно агрегируются (вывод через /), заглавный регистр корректно переносится на переведенные лексемы, а авторская пунктуация остается неизменной.

4.3 Сборка через CMake

Для автоматизации компиляции в различных средах используется система сборки CMake.

Конфигурационный файл `CMakeLists.txt`:

```
1 cmake_minimum_required(VERSION 3.10)
2 project(rpr(проект_))
3 add_executable(rgr rgr.c)
```

Команды генерации и сборки выполняются из директории `build`, что позволяет не засорять корневую папку промежуточными объектными файлами.

ЗАКЛЮЧЕНИЕ

В результате выполнения расчётно-графического задания разработана консольная утилита на языке C для контекстного перевода текста. Программа успешно прошла все этапы тестирования.

Выполненные требования (Критерий «Отлично»):

- Решение задачи на языке C с полным сохранением форматирования текста (пробелы, табы, знаки препинания);
- Реализовано динамическое масштабирование памяти словаря (`malloc/realloc`) с защитой от утечек памяти;
- Применена структура данных `DictRec` для организации информации;
- Полностью поддерживается многозначность словарных статей;
- Внедрён кроссплатформенный алгоритм обработки регистра кириллицы (UTF-8).

Разработанная программа работает стабильно, эффективно использует оперативную память и готова к практическому применению.

ПРИЛОЖЕНИЕ А. ЛИСТИНГ ПРОГРАММЫ

A.1. rgr.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <ctype.h>
5
6 typedef struct {
7     char og[256];
8     char transl[5][256];
9     int t_count;
10 } DictRec;
11
12 int make_low(const char* src, char* dst) {
13     strcpy(dst, src);
14     unsigned char b1 = dst[0], b2 = dst[1];
15
16     if (b1 >= 'A' && b1 <= 'Z') {
17         dst[0] += 32;
18         return 1;
19     }
20
21     if (b1 == 0xD0 && b2 >= 0x90 && b2 <= 0xAF) {
22         if (b2 <= 0x9F) {
23             dst[1] = b2 + 0x20;
24         } else {
25             dst[0] = 0xD1;
26             dst[1] = b2 - 0x20;
27         }
28         return 1;
29     }
30     return 0;
31 }
32
33 void fprint_upper(FILE *out, const char *str) {
34     unsigned char b1 = str[0], b2 = str[1];
35
36     if (b1 >= 'a' && b1 <= 'z') {
37         fprintf(out, "%c%s", b1 - 32, str + 1);
38     } else if (b1 == 0xD0 && b2 >= 0xB0 && b2 <= 0xBF) {
39         fprintf(out, "%c%c%s", 0xD0, b2 - 0x20, str + 2);
40     } else if (b1 == 0xD1 && b2 >= 0x80 && b2 <= 0x8F) {
41         fprintf(out, "%c%c%s", 0xD0, b2 + 0x20, str + 2);
42     } else {
43         fprintf(out, "%s", str);
44     }
```

```

45 }
46
47 DictRec* load_dict(const char *filename, int *dict_size) {
48     FILE *f = fopen(filename, "r");
49     if (!f) {
50         printf("Ошибка: файл словаря не найден\n");
51         return NULL;
52     }
53
54     int capacity = 10, count = 0;
55     DictRec *dict = malloc(capacity * sizeof(DictRec));
56
57     if (!dict) {
58         printf("Ошибка: Не удалось выделить память под словарь.\n");
59         fclose(f);
60         return NULL;
61     }
62
63     char line[1024];
64     while (fgets(line, sizeof(line), f)) {
65         char *word = strtok(line, " \n\r\t");
66         if (!word) continue;
67
68         int idx = -1;
69         for (int i = 0; i < count; i++) {
70             if (strcmp(dict[i].og, word) == 0) {
71                 idx = i;
72                 break;
73             }
74         }
75
76         if (idx == -1) {
77             if (count >= capacity) {
78                 capacity *= 2;
79                 DictRec *temp = realloc(dict, capacity * sizeof(DictRec));
80
81                 if (!temp) {
82                     printf("Ошибка: Нехватка памяти при расширении.\n");
83                     free(dict);
84                     fclose(f);
85                     return NULL;
86                 }
87                 dict = temp;
88             }
89             idx = count++;
90             strcpy(dict[idx].og, word);
91             dict[idx].t_count = 0;
92         }

```

```

93     char *trans;
94     while ((trans = strtok(NULL, " \n\r\t")) && dict[idx].t_count < 5) {
95         strcpy(dict[idx].transl[dict[idx].t_count++], trans);
96     }
97 }
98
99 fclose(f);
100
101 if (count == 0) {
102     printf("Предупреждение: Файл '%s' пуст.\n", filename);
103     free(dict);
104     return NULL;
105 }
106 *dict_size = count;
107 return dict;
108 }
109
110 void transl_word(const char *word, DictRec *dict, int dict_size, FILE *out) {
111     char search[256];
112     int is_upper = make_low(word, search);
113
114     for (int i = 0; i < dict_size; i++) {
115         if (strcmp(search, dict[i].og) == 0) {
116             for (int j = 0; j < dict[i].t_count; j++) {
117                 if (is_upper) {
118                     fprintf_upper(out, dict[i].transl[j]);
119                 } else {
120                     fprintf(out, "%s", dict[i].transl[j]);
121                 }
122
123                 if (j < dict[i].t_count - 1) {
124                     fprintf(out, "/");
125                 }
126             }
127             return;
128         }
129     }
130     fprintf(out, "%s", word);
131 }
132
133 int main(int argc, char *argv[]) {
134     if (argc != 4) {
135         printf("Используйте: %s текст<> словарь<> результат<>\n", argv[0]);
136         return 1;
137     }
138
139     int dict_size = 0;
140     DictRec *dict = load_dict(argv[2], &dict_size);

```

```

141     if (!dict) {
142         return 1;
143     }
144
145     FILE *in = fopen(argv[1], "r");
146     if (!in) {
147         printf("Ошибка: Не удалось открыть '%s'.\n", argv[1]);
148         free(dict);
149         return 1;
150     }
151
152     FILE *out = fopen(argv[3], "w");
153     if (!out) {
154         printf("Ошибка: Не удалось создать '%s'.\n", argv[3]);
155         fclose(in);
156         free(dict);
157         return 1;
158     }
159
160     char token[256];
161     int pos = 0;
162     int ch;
163
164     while ((ch = fgetc(in)) != EOF) {
165         if (isalpha((unsigned char)ch) || (unsigned char)ch >= 128) {
166             if (pos < 255) token[pos++] = ch;
167         } else {
168             if (pos > 0) {
169                 token[pos] = '\0';
170                 transl_word(token, dict, dict_size, out);
171                 pos = 0;
172             }
173             fputc(ch, out);
174         }
175     }
176
177     if (pos > 0) {
178         token[pos] = '\0';
179         transl_word(token, dict, dict_size, out);
180     }
181
182     free(dict);
183     fclose(in);
184     fclose(out);
185
186     printf("Успех: Результат сохранен в '%s'.\n", argv[3]);
187     return 0;
188 }

```

A.2. CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.10)
2
3 project(prp(проект_))
4 add_executable(rgr rgr.c)
```